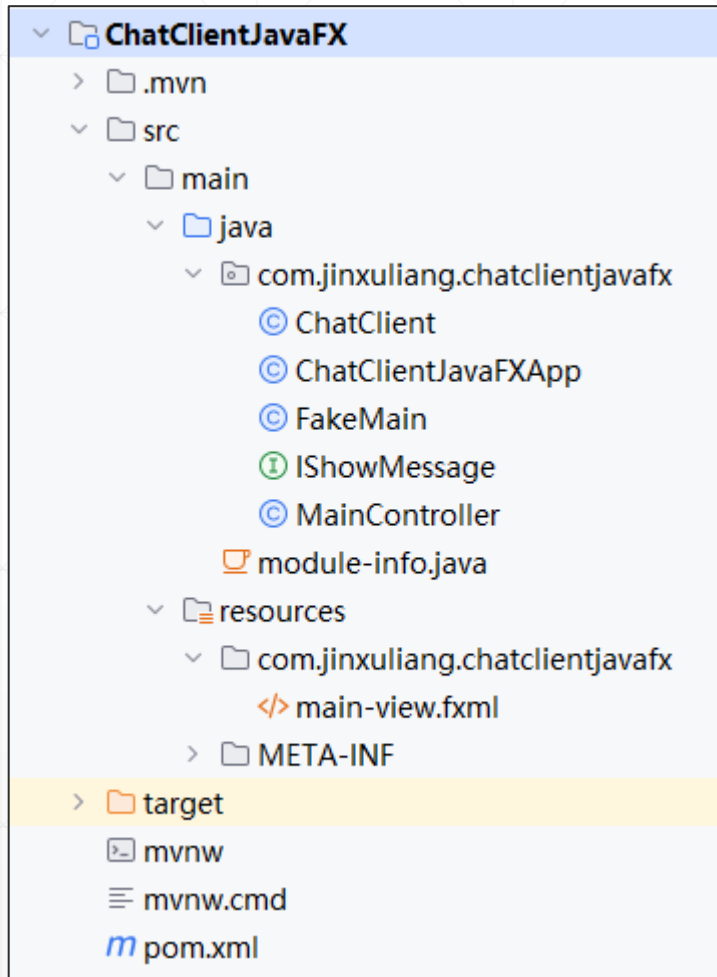


JavaFX客户端分析

北京理工大学计算机学院
金旭亮

项目架构



JavaFX客户端使用MVC架构，用户交互功能集中于MainController类中。

ChatClientJavaFXApp是JavaFX的真正入口点，而FakeMain则是一个对上述入口点的一个“封装”，主要目的是为了生成能直接双击运行的jar包。

聊天的主要功能代码，集中于ChatClient类中，它通过IShowMessage接口，回调MainController所定义的方法，用于更新界面。

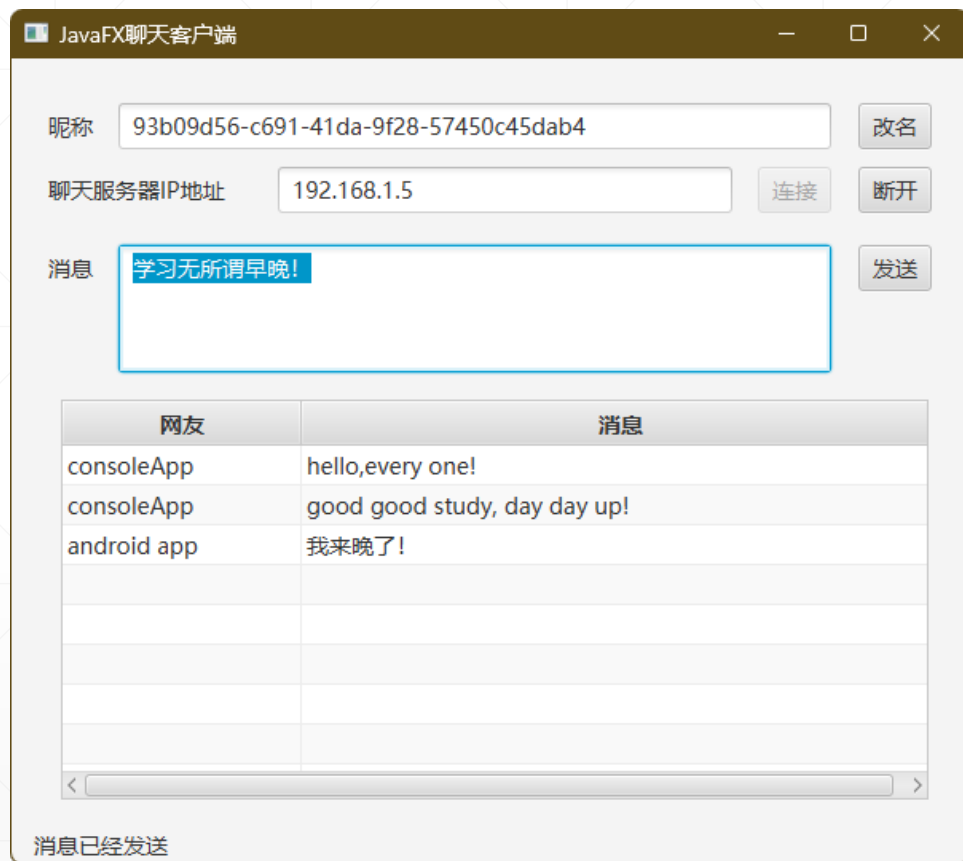
程序入口点

```
public class FakeMain {  
    public static void main(String[] args) {  
        ChatClientJavaFXApp.main(args);  
    }  
}
```



```
public class ChatClientJavaFXApp extends Application {  
    @Override  
    public void start(Stage stage) throws IOException {  
        FXMLLoader fxmlLoader = new FXMLLoader(  
            ChatClientJavaFXApp.class.getResource("main-view.fxml"));  
        Parent root = fxmlLoader.load();  
        MainController controller = fxmlLoader.getController();  
        Scene scene = new Scene(root);  
        stage.setTitle("JavaFX聊天客户端");  
        stage.setScene(scene);  
        //关闭主窗体时,退出可能正在运行的线程  
        stage.setOnCloseRequest(e -> {  
            if (controller != null) {  
                controller.quit();  
            }  
        });  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch();  
    }  
}
```

界面设计



//控制器实现了IShowMessage接口

```
public class MainController  
    implements Initializable, IShowMessage {
```

> 控件变量说明

6 usages

```
private ChatClient chatClient = null;
```

5 usages

```
private String userName;
```

//用于保存聊天消息，它们将显示在TableView中

2 usages

```
ObservableList<ChatMessage> messages =  
    FXCollections.observableArrayList();
```

信息的显示

```
1 implementation
public interface IShowMessage {
    //在表格中显示消息
    1 usage 1 implementation
    void showMessage(ChatMessage message);
    //用于显示状态信息
    3 usages 1 implementation
    void showStatus(String message);
}
```

```
1 usage
@Override
public void showMessage(ChatMessage message) {
    //利用数据绑定显示聊天消息
    messages.add(ChatMessage.builder().
        userName(message.getUserName())
        .message(message.getMessage())
        .build());
}

3 usages
@Override
public void showStatus(String message) {
    //跨线程访问UI控件, 更新状态信息
    Platform.runLater(() -> {
        this.lblStatus.setText(message);
    });
}
```

MainController.java

连接服务器的功能代码

```
btnConnect.setOnAction(e -> {  
    String ip = txtIP.getText();  
    if (!ip.isEmpty()) {  
        new Thread(() -> {  
            //实例化一个ChatClient, 用于接收和发送消息  
            chatClient = new ChatClient(txtIP.getText(),  
                uiController: this, userName);  
            chatClient.start();  
        }).start();  
        enableControlsAfterConnect();  
        txtMessage.requestFocus();  
    }  
});
```

消息的发送

```
btnSend.setOnAction(e -> {  
    if (chatClient == null) {  
        return;  
    }  
    String message = txtMessage.getText();  
    if (!message.isEmpty()) {  
        new Thread(() -> {  
            try {  
                //将用户名与消息序列化为Json,发送出去  
                ChatMessage messageToBeSent = ChatMessage.builder()  
                    .userName(userName)  
                    .message(message)  
                    .build();  
                chatClient.send(ChatMessage.toJson(messageToBeSent));  
                Platform.runLater(() -> {  
                    this.lblStatus.setText("消息已经发送");  
                    this.txtMessage.selectAll();  
                    this.txtMessage.requestFocus();  
                });  
            } catch (IOException ex) {  
                throw new RuntimeException(ex);  
            }  
        }).start();  
    }  
});
```

ChatClient-数据成员

```
public class ChatClient {  
    5 usages  
    private Socket socket;  
    //接收消息  
    2 usages  
    private BufferedReader reader;  
    //发送消息  
    5 usages  
    private BufferedWriter writer;  
    4 usages  
    private IShowMessage uiController;  
    2 usages  
    private String serverIP;  
    2 usages  
    private String userName;  
  
    1 usage  
    public ChatClient(String serverIP, IShowMessage uiController, String userName) {  
        this.serverIP = serverIP;  
        this.uiController = uiController;  
        this.userName = userName;  
    }  
}
```


消息的发送与接收

// 发送消息给服务器

2 usages

```
public void send(String msg) throws IOException {  
    if (!socket.isOutputShutdown()) {  
        writer.write(str: msg + "\n");  
        writer.flush();  
    }  
}
```

// 从服务器接收消息

1 usage

```
public String receive() {  
    String msg = null;  
    if (!socket.isInputShutdown()) {  
        try {  
            msg = reader.readLine();  
        } catch (IOException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
    return msg;  
}
```

ChatClient-用户下线

```
// 关闭Socket并退出
2 usages
public void close() {
    if (writer != null) {
        try {
            this.uiController.showStatus("关闭socket");
            //发送关闭消息
            ChatMessage quitMessage = ChatMessage.builder().userName(userName)
                .message("quit").build();
            send(ChatMessage.toJson(quitMessage));
            writer.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

ChatClient-主控流程

```
1 usage
public void start() {
    try {
        // 创建socket
        socket = new Socket(serverIP, Constants.DEFAULT_SERVER_PORT);
        // 创建IO流
        reader = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );
        writer = new BufferedWriter(
            new OutputStreamWriter(socket.getOutputStream())
        );
        uiController.showStatus("正在等待服务端传回的消息...");
        // 读取服务器转发的消息
        String msg = null;
        while ((msg = receive()) != null) {
            ChatMessage message = ChatMessage.fromJson(msg);
            uiController.showMessage(message);
        }
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        close();
    }
}
```